

Lab no 5

Date : 11/02/2026

Programs on arrays in CUDA

Name: Satyam Singh

Reg no: 230905256

Roll No: 37

CSE B2

Lab exercise:

Sample Question) program to add two numbers

Code)

```
#include<stdio.h>
```

```
#include"cuda_runtime.h"
```

```
#include"device_launch_parameters.h"
```

```
__global__ void add(int*a,int*b,int*c){
```

```
*c = *a+*b;
```

```
}
```

```
int main(){
```

```
int a,b,c;
```

```
int*d_a,*d_b,*d_c;
```

```
int size = sizeof(int);
```

```
cudaMalloc((void**)&d_a,size);
cudaMalloc((void**)&d_b,size);
cudaMalloc((void**)&d_c,size);
a=3;
b=5;
cudaMemcpy(d_a,&a,size,cudaMemcpyHostToDevice);
cudaMemcpy(d_b,&b,size,cudaMemcpyHostToDevice);
add<<<1,1>>>(d_a,d_b,d_c);
cudaMemcpy(&c,d_c,size,cudaMemcpyDeviceToHost);
printf("Result: %d\n",c);
cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);
return 0;
}
```

Output:

```
● STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ nvcc sample.cu -o sample
● STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ ./sample
Result: 8
```

Q1 a) Write a program in CUDA to add two vectors of length N using block size as N

Ans)

```
#include<stdio.h>
```

```
#include"cuda_runtime.h"
```

```
#include"device_launch_parameters.h"
```

```
__global__ void vectorAdd(float *A, float *B, float *C, int N)
```

```
{
```

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
if (i < N)
```

```
C[i] = A[i] + B[i];
```

```
}
```

```
int main()
```

```
{
```

```
int N;
```

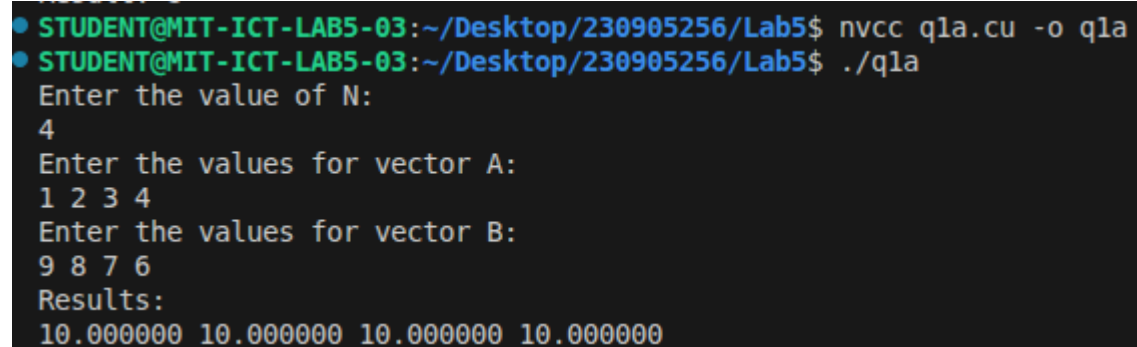
```
printf("Enter the value of N: \n");
```

```
scanf("%d",&N);
```

```
float *h_A, *h_B, *h_C;
float *d_A, *d_B, *d_C;
int size = N * sizeof(float);
h_A = (float*)malloc(size);
h_B = (float*)malloc(size);
h_C = (float*)malloc(size);
printf("Enter the values for vector A:\n");
for(int i = 0;i<N;i++){
scanf("%f",&h_A[i]);
}
printf("Enter the values for vector B:\n");
for(int i = 0;i<N;i++){
scanf("%f",&h_B[i]);
}
cudaMalloc((void**)&d_A, size);
cudaMalloc((void**)&d_B, size);
cudaMalloc((void**)&d_C, size);
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size,
cudaMemcpyHostToDevice);
vectorAdd<<<N, 1>>>(d_A, d_B, d_C, N);
```

```
cudaMemcpy(h_C, d_C, size,  
cudaMemcpyDeviceToHost);  
  
printf("Results:\n");  
  
for (int i = 0; i < N; i++)  
    printf("%f ", h_C[i]);  
  
printf("\n");  
  
free(h_A);  
  
free(h_B);  
  
free(h_C);  
  
cudaFree(d_A);  
  
cudaFree(d_B);  
  
cudaFree(d_C);  
  
return 0;  
  
}
```

Output:



```
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ nvcc q1a.cu -o q1a  
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ ./q1a  
Enter the value of N:  
4  
Enter the values for vector A:  
1 2 3 4  
Enter the values for vector B:  
9 8 7 6  
Results:  
10.000000 10.000000 10.000000 10.000000
```

Q1 b) Write a program in CUDA to add two vectors of length N using N threads.

Ans)

```
#include<stdio.h>
```

```
#include"cuda_runtime.h"
```

```
#include"device_launch_parameters.h"
```

```
__global__ void vectorAdd(float *A, float *B, float *C, int N)
```

```
{
```

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
if (i < N)
```

```
C[i] = A[i] + B[i];
```

```
}
```

```
int main()
{
    int N;
    printf("Enter the value of N: \n");
    scanf("%d",&N);
    float *h_A, *h_B, *h_C;
    float *d_A, *d_B, *d_C;
    int size = N * sizeof(float);
    h_A = (float*)malloc(size);
    h_B = (float*)malloc(size);
    h_C = (float*)malloc(size);
    printf("Enter the values for vector A:\n");
    for(int i = 0;i<N;i++){
        scanf("%f",&h_A[i]);
    }
    printf("Enter the values for vector B:\n");
    for(int i = 0;i<N;i++){
        scanf("%f",&h_B[i]);
    }
    cudaMalloc((void**)&d_A, size);
```

```
cudaMalloc((void**)&d_B, size);
cudaMalloc((void**)&d_C, size);
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size,
cudaMemcpyHostToDevice);
vectorAdd<<<1,N>>>(d_A, d_B, d_C, N);
cudaMemcpy(h_C, d_C, size,
cudaMemcpyDeviceToHost);
printf("Results:\n");
for (int i = 0; i < N; i++)
printf("%f ", h_C[i]);
printf("\n");
free(h_A);
free(h_B);
free(h_C);
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
return 0;
}
```


Output:

```
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ nvcc q1b.cu -o q1b
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ ./q1b
Enter the value of N:
3
Enter the values for vector A:
1 2 3
Enter the values for vector B:
8 7 6
Results:
9.000000 9.000000 9.000000
```

Q2) Implement a CUDA program to add two vectors of length N by keeping the number of threads per block as 256(constant) and vary the number of blocks to handle N elements.

Ans)

```
#include<stdio.h>
```

```
#include"cuda_runtime.h"
```

```
#include"device_launch_parameters.h"
```

```
// Kernel
```

```
__global__ void vectorAdd(float *A, float *B, float *C, int N)
```

```
{
```

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
if (i < N)
```

```
    C[i] = A[i] + B[i];
```

```
}
```

```
int main()
```

```
{
```

```
    int N;
```

```
    printf("Enter the value of N: \n");
```

```
    scanf("%d",&N);
```

```
    float *h_A, *h_B, *h_C;
```

```
    float *d_A, *d_B, *d_C;
```

```
    int size = N * sizeof(float);
```

```
    h_A = (float*)malloc(size);
```

```
    h_B = (float*)malloc(size);
```

```
    h_C = (float*)malloc(size);
```

```
    printf("Enter the values for vector A:\n");
```

```
    for(int i = 0;i<N;i++){
```

```
        scanf("%f",&h_A[i]);
```

```
    }
```

```
    printf("Enter the values for vector B:\n");
```

```
for(int i = 0;i<N;i++){
scanf("%f",&h_B[i]);
}

cudaMalloc((void**)&d_A, size);
cudaMalloc((void**)&d_B, size);
cudaMalloc((void**)&d_C, size);
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size,
cudaMemcpyHostToDevice);
vectorAdd<<<ceil(N/256.0), 256>>>(d_A, d_B, d_C, N);
cudaMemcpy(h_C, d_C, size,
cudaMemcpyDeviceToHost);
printf("Results:\n");
for (int i = 0; i < N; i++)
printf("%f ", h_C[i]);
printf("\n");
free(h_A);
free(h_B);
free(h_C);
cudaFree(d_A);
cudaFree(d_B);
```

```
cudaFree(d_C);
```

```
return 0;
```

```
}
```

Output:

```
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ nvcc q2.cu -o q2
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ ./q2
Enter the value of N:
5
Enter the values for vector A:
1 2 3 4 5
Enter the values for vector B:
9 8 7 6 5
Results:
10.000000 10.000000 10.000000 10.000000 10.000000
```

Q3) Write a program in CUDA to process a 1D array containing angles in radians to generate sine of the angles in the output array. Use appropriate function.

Ans)

```
#include<stdio.h>
```

```
#include"cuda_runtime.h"
```

```
#include"device_launch_parameters.h"
```

```
__global__ void computeSine(float *A, float *B, int N)
```

```
{
```

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
if (i < N)
```

```
B[i] = sinf(A[i]);
```

```
}
```

```
int main()
```

```
{
```

```
int N;
```

```
printf("Enter the value of N: \n");
```

```
scanf("%d",&N);
```

```
float *h_A, *h_B;
```

```
float *d_A, *d_B;
```

```
int size = N * sizeof(float);
```

```
h_A = (float*)malloc(size);
```

```
h_B = (float*)malloc(size);
```

```
printf("Enter the values for vector A:\n");
```

```
for(int i = 0;i<N;i++){
```

```
scanf("%f",&h_A[i]);
```

```
}
```

```
cudaMalloc((void**)&d_A, size);
```

```
cudaMalloc((void**)&d_B, size);
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
computeSine<<<ceil(N/256.0),256>>>(d_A, d_B, N);
cudaMemcpy(h_B, d_B, size,
cudaMemcpyDeviceToHost);
printf("angle(radian) Sine \n");
for (int i = 0; i < N; i++)
printf("%f %f \n", h_A[i],h_B[i]);
printf("\n");
free(h_A);
free(h_B);
cudaFree(d_A);
cudaFree(d_B);
return 0;
}
```

Output

```
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ nvcc q3.cu -o q3
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab5$ ./q3
Enter the value of N:
3
Enter the values for vector A:
0 1.57 3.14
angle(radian)      Sine
0.000000           0.000000
1.570000           1.000000
3.140000           0.001593
```